NOV 5th research:

The brain can "reuse" or project previously seen visual information into the blind field of hemianopia patients, a phenomenon driven by cortical "filling-in" and visual release mechanisms. When a stroke or injury cuts off live sensory signals to one half of the visual cortex, the affected area becomes starved of input. To compensate for this sudden sensory void, the brain's predictive systems and memory networks can project unformed shapes, textures, or even fully intact, previously seen objects into the blind zone

The patient’s internal model of the blind side is built from whatever they last looked at, and it can’t update unless they look there again. The brain keeps its spatial memory of a scene even when it can’t see part of it, and that memory can go out of date.

Also, the blind side of patients doesn't really look like a black screen, its just a discontinuity in the visual scene.

Pipeline:

Milestone 0: Get gaze into Unity (day 1-2)

Build:

- In the Meta XR SDK (your repo already uses it), enable eye tracking in the OVRManager settings and request the eye tracking permission at runtime. The Quest Pro also needs eye tracking turned on and calibrated in the headset’s own settings.
- Add the SDK’s eye gaze component to each eye and draw a small reticle where the gaze ray hits a plane or sphere about 2 m away.
- Check whether it works over Quest Link in the editor. If it does, iteration is much faster than building to the headset every time.

Pass: the reticle follows your eyes (not your head), and you can see it freeze or jump when you blink.

Trials and their purpose for the block:

Test	What the block does	What it tells you
Saccade steps	Jumps between fixed points (horizontal, vertical, diagonal), pausing 1 to 2 s at each	Landing accuracy and saccade timing
Smooth pursuit, square	Glides along a square at constant speed (try 10, 20°/s)	Tracking quality on horizontal and vertical segments
Smooth pursuit, "X"	Glides along both diagonals through the center	Diagonal behavior and corner handling
Return to center	Jumps back to center between sweeps	Resets fixation, gives clean baselines

The goal of the moving block test is to measure how far behind the eye tracker is. The trouble is that "behind" has two possible sources, and the test has to separate them.

The problem with saccade steps

Say the block jumps to a new spot at time 0, and your logged gaze starts moving at 220 ms. That 220 ms is made of two delays added together:

1. **Human delay:** the person has to notice the jump, decide to look, and send the signal to the eye muscles. This takes roughly 150 to 250 ms and varies by person and by trial.
2. **System delay:** the time between the eye actually moving and Unity receiving that movement (tracker processing, data transfer, your code).

You only care about the second one, but the number you see is the sum, and you can't pull the two apart. If the system delay is 30 ms and the human delay is 190 ms, you just see 220. A change in the human part (the person is tired, or expecting the jump) would look like a change in tracker lag

How to work around that:

1. Use smooth pursuit for the lag number (the main fix). When someone follows a predictable, steadily moving block, they aren't reacting to a surprise, so human delay mostly disappears. Slide the gaze trace against the block trace until they match, and that shift is mostly system lag. Do this for several people and several passes, and use the median.

2. Make the saccade steps predictable too. If the block jumps at a regular rhythm between known spots (say every 1.5 s, back and forth between two points) and the person knows the pattern, they start moving their eyes right as the block jumps, or even slightly before. Human delay shrinks toward zero. You can then compare timing the same way as in pursuit. This is a useful cross-check on the pursuit number.

3. Read the timestamp the tracker gives you. The combined-gaze API I mentioned reports a timestamp with each sample. Compare it to the time Unity processes that sample. The gap is how old the data is by the time your code sees it. This measures part of the system delay directly, with no humans involved. It doesn't include the tracker's internal processing or the display delay, so treat it as a lower bound, not the full story.

What kind of data are we looking for?